BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

Facultad de Ciencias de la Computación

Red de Amigos

Sugerencia de amistades con grafos en Racket
Lenguajes de Programación

Integrantes:

Hugo Javier Rodriguez Bravo
Josue Francisco Hernandez Iturbide
Miguel Da Silva Sanabria
Carlos Daniel Macuil Carcaño
Sahara Ximena Cuacuas Luna

Profesora: Meliza Contreras Gonzalez

NRC: 25480

Otoño 2026

1. Introducción

En esta práctica se resuelve un problema típico de estructuras de datos empleando el paradigma funcional. El lenguaje seleccionado por el equipo es Racket, un lenguaje de la familia Lisp/Scheme. El problema elegido es la sugerencia de amistades dentro de una red social, el cual se modela con un grafo. La solución se construyó únicamente con funciones recursivas, listas, funciones de orden superior y composición de funciones, sin utilizar ciclos, y permite al usuario ingresar sus propios datos desde un menú interactivo.

2. Definición del problema

Se requiere modelar una red social como un grafo no dirigido, donde los nodos representan personas y las aristas representan amistades. El sistema debe permitir agregar y eliminar personas, crear y romper amistades, y generar sugerencias de amistad para una persona a partir de los amigos de sus amigos, ordenadas por el número de amigos en común.

En términos de grafos, sugerir amigos a una persona consiste en encontrar los nodos que se encuentran a distancia 2 de ella (los vecinos de sus vecinos) y que todavía no son sus vecinos directos. Entre más amigos en común tenga una persona sugerida, más probable es que ambas se conozcan, por lo que las sugerencias se ordenan de mayor a menor según ese criterio. Si la persona no tiene amigos de amigos, por ejemplo porque acaba de ser agregada a la red sin conexiones, el sistema le sugiere a todas las personas de la red que aún no son sus amigas.

2.1. Representación del grafo

El grafo se representa con una lista de adyacencia construida solamente con listas. Cada nodo es una lista cuyo primer elemento es el nombre de la persona y cuyos elementos restantes son sus amigos. La red completa es una lista de nodos. Como la amistad es mutua, cada arista aparece en los dos nodos que conecta. Una persona sin amigos se representa con una lista de un solo elemento. El programa inicia con la siguiente red precargada, que se muestra gráficamente en la Figura 1.

Código 1: Red precargada

```
(define red-inicial
  '(("ana" "beto" "carla")
    ("beto" "ana" "diego")
    ("carla" "ana" "diego" "elena")
    ("diego" "beto" "carla" "fer")
    ("elena" "carla")
    ("fer" "diego")
    ("gael")))
```

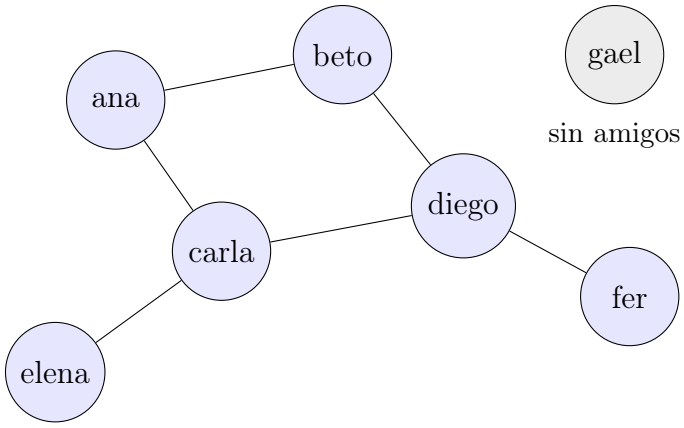


Figura 1: Grafo de la red precargada

2.2. Operaciones del sistema

El programa funciona mediante un menú con las opciones que se describen en la Tabla 1.

Opción	**Nombre**	**Descripción**
1	Mostrar red	Muestra a cada persona con su lista de amigos.
2	Agregar persona	Agrega un nodo nuevo y pregunta de quién es amigo; si no se elige a nadie, el nodo queda aislado.
3	Sugerir amigos	Muestra los amigos de los amigos de una persona, ordenados por amigos en común, y permite crear una nueva amistad con alguno de ellos.
4	Romper amistad	Elimina la arista entre dos personas.
5	Eliminar persona	Elimina el nodo de una persona junto con todas sus aristas.
6	Salir	Termina el programa.

Cuadro 1: Opciones del menú

3. Justificación del uso del paradigma funcional

El problema se adapta de forma natural al paradigma funcional por varias razones. En primer lugar, el grafo se puede representar completamente con listas, que son la estructura de datos central de los lenguajes funcionales, sin necesidad de apuntadores ni de estructuras mutables. Recorrer una lista de adyacencia es un proceso recursivo por naturaleza: se revisa el primer nodo y se repite el mismo proceso con el resto de la lista hasta llegar a la lista vacía.

En segundo lugar, las operaciones sobre la red se expresan como transformaciones de datos. Obtener los nombres de todas las personas es aplicar una función a cada nodo (`map`), descartar a quienes ya son amigos es un filtrado (`filter`), y generar las sugerencias es una secuencia de pasos donde la salida de uno es la entrada del siguiente, lo cual corresponde directamente a la composición de funciones.

En tercer lugar, la inmutabilidad simplifica el manejo del estado de la red. Ninguna operación modifica la red existente; cada una regresa una red nueva con el cambio aplicado. Esto elimina los errores típicos de estado compartido, como modificar una amistad en un nodo y olvidar actualizarla en el otro a mitad de la operación, y permite probar cada función de forma aislada en el intérprete con cualquier red de entrada. Finalmente, al tratar a las funciones como datos, el menú mismo puede representarse como una lista que asocia cada opción con la función que la atiende.

4. Diseño de la solución

A continuación se describen las funciones más representativas del programa. El código completo se encuentra en el Anexo A.

4.1. Consultas básicas y recursión

Las funciones `nombre` y `amigos-de` dan un significado a las partes de un nodo. La función `buscar` recorre la red de manera recursiva: si la red está vacía regresa `#f`, si el primer nodo corresponde a la persona buscada lo regresa, y en otro caso continúa la búsqueda con el resto de la red. La función `amigos` utiliza `buscar` y siempre regresa una lista, que es vacía cuando la persona no existe.

Código 2: Consultas básicas

```
(define (nombre nodo) (car nodo))
(define (amigos-de nodo) (cdr nodo))

(define (buscar red persona)
  (cond [(null? red) #f]
        [(string=? (nombre (car red)) persona) (car red)]
        [else (buscar (cdr red) persona)]))

(define (amigos red persona)
  (let ([nodo (buscar red persona)])
    (if nodo (amigos-de nodo) '())))

(define (personas red) (map nombre red))
```

4.2. Modificación de la red: orden superior y composición

La función `actualizar` es de orden superior porque recibe como parámetro una función `f`, que se aplica a la lista de amigos de una persona. La función `modificar-lazo` también es de orden superior, pero en el otro sentido: regresa una función que, dada una red, aplica una operación a la lista de amigos de una persona. Con ellas, `conectar` y `desconectar` se definen como la composición de dos modificaciones, una para cada extremo de la arista, lo cual garantiza que el grafo siga siendo no dirigido.

Código 3: Actualización de la red con funciones de orden superior

```
(define (actualizar red persona f)
  (map (lambda (nodo)
         (if (string=? (nombre nodo) persona)
             (cons (nombre nodo) (f (amigos-de nodo)))
             nodo))
       red))

(define (modificar-lazo operacion a b)
  (lambda (red) (actualizar red a (lambda (lista) (operacion lista b)))))

(define (conectar red a b)
  ((compose (modificar-lazo agregar-a b a) (modificar-lazo agregar-a a b))
      red))

(define (desconectar red a b)
  ((compose (modificar-lazo quitar-de b a) (modificar-lazo quitar-de a b))
      red))
```

De forma similar, eliminar a una persona es la composición de dos pasos: quitar su nodo de la red y quitar su nombre de las listas de amigos de todos los demás.

Código 4: Eliminación de una persona

```
(define (eliminar-persona red persona)
  ((compose (sin-referencias persona) (sin-nodo persona)) red))
```

4.3. Sugerencia de amigos

La función `amigos-de-amigos` utiliza `append-map` para obtener, en una sola lista, los amigos de cada uno de los amigos de la persona. Sobre esa lista se aplica la composición de tres funciones: descartar a la propia persona y a quienes ya son sus amigos, eliminar repetidos y ordenar por número de amigos en común. Cuando el resultado es vacío, `sugerir` regresa a todas las personas de la red que aún no son sus amigas.

Código 5: Generación de sugerencias

```
(define (amigos-de-amigos red persona)
  (append-map (lambda (amigo) (amigos red amigo)) (amigos red persona)))

(define (sugerencias-directas red persona)
  ((compose (ordenar-por-comun red persona)
            remove-duplicates
            (descartar-conocidos red persona))
   (amigos-de-amigos red persona)))

(define (sugerir red persona)
  (let ([directas (sugerencias-directas red persona)])
    (if (null? directas)
        ((descartar-conocidos red persona) (personas red))
        directas)))
```

4.4. Menú recursivo y entrada de datos

Para cumplir con la restricción de no utilizar ciclos, el menú es una función recursiva que recibe la red actual, ejecuta la opción elegida y se llama a sí misma con la red resultante. Las opciones se guardan en una lista que asocia cada número con su función, de modo que las funciones se manejan como datos. La entrada del usuario se normaliza con la composición `(compose string-downcase string-trim)`.

Código 6: Menú recursivo

```
(define opciones
  (list (cons "1" opcion-mostrar)
        (cons "2" opcion-agregar)
        (cons "3" opcion-sugerir)
        (cons "4" opcion-romper)
        (cons "5" opcion-eliminar)))

(define (menu red)
  ...
  (let* ([opcion (leer "Elige una opción: ")]
         [accion (assoc opcion opciones)])
    (cond [(string=? opcion "6") (displayln "¡Hasta luego!")]
          [accion (menu ((cdr accion) red))]
          [else (displayln "Opción no válida.") (menu red)])))
```

5. Tipo de evaluación en Racket

Racket realiza una evaluación estricta en **orden aplicativo**, con paso de parámetros por **llamada por valor**. Esto significa que, antes de aplicar una función, se evalúan todos sus argumentos (de izquierda a derecha) hasta obtener valores, y son esos valores los que se sustituyen en el cuerpo de la función. En contraste, en el orden normal o con llamada por nombre, los argumentos se sustituyen sin evaluar y solo se evalúan cuando el cuerpo de la función los necesita.

Una excepción importante son las formas especiales como `if`, `cond`, `and`, `or`, `let` y `lambda`. Estas no son funciones, por lo que no evalúan todos sus componentes de antemano: `if` y `cond` evalúan primero la condición y después únicamente la rama elegida. Racket también ofrece el dialecto `#lang lazy` con evaluación perezosa (llamada por necesidad), pero en este proyecto se utilizó `#lang racket`, que es estricto.

5.1. Comprobación del orden aplicativo

La siguiente función regresa su primer argumento e ignora el segundo:

Código 7: Prueba del orden aplicativo

```
(define (primero a b) a)
(primero 1 (/ 1 0))
```

En orden normal, la expresión `(/ 1 0)` nunca se evaluaría porque `b` no se usa en el cuerpo, y el resultado sería `1`. En Racket, en cambio, se produce el error `/: division by`

`zero`, porque ambos argumentos se evalúan antes de aplicar la función. Esto confirma que Racket utiliza orden aplicativo.

5.2. Ejemplo: `son-amigos?`

Código 8: Función son-amigos?

```
(define (son-amigos? red a b)
  (if (member b (amigos red a)) #t #f))
```

La evaluación de `(son-amigos? red-inicial .ªnabeto")` procede de la siguiente manera:

Código 9: Traza de evaluación de son-amigos?

```
(son-amigos? red-inicial "ana" "beto")
→ se evalúan los argumentos: red-inicial es la lista de nodos,
  "ana" y "beto" ya son valores
→ se sustituyen en el cuerpo:
  (if (member "beto" (amigos red-inicial "ana")) #t #f)
→ if evalúa primero la condición; member es una función,
  así que primero se evalúa su argumento:
  (amigos red-inicial "ana") → '("beto" "carla")
→ (member "beto" '("beto" "carla")) → '("beto" "carla")
→ la condición es verdadera (distinta de #f) → #t
```

La llamada a `amigos` se resuelve por completo antes de que `member` se aplique, y `member` recibe la lista ya calculada, no la expresión que la produce.

5.3. Ejemplo: `conectar`

En `(conectar red-inicial .ªnadiego")`, primero se evalúan los argumentos de `compose`: cada llamada a `modificar-lazo` regresa una función (una clausura que recuerda la operación y los dos nombres). Después, `compose` regresa una nueva función, y esta se aplica a la red. Al aplicarla, la función de la derecha se ejecuta primero y produce una red donde ana ya tiene a diego como amigo; esa red, ya calculada, se pasa como valor a la función de la izquierda, que agrega a ana en la lista de diego. En ningún momento se pasa una expresión pendiente de evaluar: cada paso recibe el valor que produjo el anterior.

5.4. Ejemplo: `buscar`

En la llamada recursiva `(buscar (cdr red) persona)`, la expresión `(cdr red)` se evalúa antes de hacer la llamada, de modo que cada llamada recibe una lista concreta, más corta que la anterior. Además, `buscar` aprovecha que `cond` es una forma especial: la condición `(null? red)` se evalúa primero y, si es verdadera, las demás cláusulas no se evalúan. Gracias a esto nunca se intenta obtener el `car` de una lista vacía. Como la llamada recursiva es lo último que hace la función, se trata de una recursión de cola, que Racket ejecuta sin acumular llamadas pendientes.

6. Funcionamiento del sistema

6.1. Ejemplo 1: sugerencias con datos precargados

Se eligió la opción 3 para sugerir amigos a ana. Sus amigos son beto y carla, por lo que el sistema sugiere a diego, que tiene dos amigos en común con ella, y a elena, que tiene uno. Se aceptó la sugerencia de diego y, con la opción 1, se comprobó que la nueva amistad aparece en los nodos de ambos.

```
Welcome to DrRacket, version 9.3 [cs].
Language: racket; memory limit: 128 MB.

========= RED DE AMIGOS =========
1. Mostrar red
2. Agregar persona
3. Sugerir amigos
4. Romper amistad
5. Eliminar persona
6. Salir
Elige una opción: 3
¿A quién le sugerimos amigos?: ana
Amigos de los amigos de ana:
  1. diego (2 amigo(s) en común)
  2. elena (1 amigo(s) en común)
Número de la persona a agregar (Enter para no agregar): 1
¡ana y diego ahora son amigos!

========= RED DE AMIGOS =========
1. Mostrar red
2. Agregar persona
3. Sugerir amigos
4. Romper amistad
5. Eliminar persona
6. Salir
Elige una opción: 1

--- Red de amigos ---
  ana -> beto, carla, diego
  beto -> ana, diego
  carla -> ana, diego, elena
  diego -> beto, carla, fer, ana
  elena -> carla
  fer -> diego
  gael -> (sin amigos)

========= RED DE AMIGOS =========
1. Mostrar red
2. Agregar persona
3. Sugerir amigos
4. Romper amistad
5. Eliminar persona
6. Salir
Elige una opción:
```

Figura 2: Sugerencias para ana y creación de la amistad con diego

6.2. Ejemplo 2: nueva persona sin amigos

Con la opción 2 se agregó a una nueva persona, hugo, sin elegir ninguna amistad, por lo que su nodo quedó aislado. Al pedir sugerencias para hugo, el sistema indicó que no tiene amigos de amigos y mostró a todas las personas de la red. Se eligió a diego y, al volver a pedir sugerencias, el sistema ya mostró a los amigos de diego (beto, carla y fer), cada uno con un amigo en común.

```
1 #lang racket
```

```
1. Mostrar red
2. Agregar persona
3. Sugerir amigos
4. Romper amistad
5. Eliminar persona
6. Salir
Elige una opción: 2
Nombre de la nueva persona: hugo
Personas en la red: ana, beto, carla, diego, elena, fer, gael
  ¿De quién es amigo? (Enter para terminar):

========= RED DE AMIGOS =========
1. Mostrar red
2. Agregar persona
3. Sugerir amigos
4. Romper amistad
5. Eliminar persona
6. Salir
Elige una opción: 3
¿A quién le sugerimos amigos?: hugo
No tiene amigos de amigos, estas son todas las personas de la red:
  1. ana (0 amigo(s) en común)
  2. beto (0 amigo(s) en común)
  3. carla (0 amigo(s) en común)
  4. diego (0 amigo(s) en común)
  5. elena (0 amigo(s) en común)
  6. fer (0 amigo(s) en común)
  7. gael (0 amigo(s) en común)
Número de la persona a agregar (Enter para no agregar): 4
¡hugo y diego ahora son amigos!

========= RED DE AMIGOS =========
1. Mostrar red
2. Agregar persona
3. Sugerir amigos
4. Romper amistad
5. Eliminar persona
6. Salir
Elige una opción: 3
¿A quién le sugerimos amigos?: hugo
Amigos de los amigos de hugo:
  1. beto (1 amigo(s) en común)
  2. carla (1 amigo(s) en común)
  3. fer (1 amigo(s) en común)
Número de la persona a agregar (Enter para no agregar):

========= RED DE AMIGOS =========
```

Figura 3: Nueva persona sin amigos y sugerencias antes y después de su primera amistad

7. Conclusiones

El desarrollo de este proyecto permitió comprobar que el paradigma funcional es adecuado para resolver problemas sobre estructuras de datos recursivas como las listas y los grafos. Entre sus principales ventajas se encontró que el código resulta conciso y cercano a la definición matemática del problema: la sugerencia de amigos se escribe prácticamente como su descripción, una composición de pasos sobre los amigos de los amigos. La inmutabilidad evitó errores de estado, ya que cada operación produce una red nueva y la anterior permanece intacta, lo que además facilitó probar cada función de forma aislada en el intérprete. Las funciones de orden superior como `map`, `filter` y `append-map` eliminaron la necesidad de escribir recorridos manuales, y la composición permitió construir operaciones complejas a partir de funciones pequeñas y reutilizables.

Entre las desventajas, el paradigma exige una forma de pensar distinta a la imperativa, y resolver todo con recursión en lugar de ciclos implica una curva de aprendizaje, al igual que la sintaxis basada en paréntesis de Racket. En cuanto a la eficiencia, cada modificación reconstruye la lista de la red en lugar de cambiar un solo valor, y las búsquedas sobre listas son lineales, lo que podría volverse costoso en redes muy grandes frente a estructuras

mutables como las tablas hash. Por último, la interacción con el usuario (leer datos e imprimir resultados) es inherentemente un efecto secundario, por lo que esa parte del programa se aleja del estilo puramente funcional y debe mantenerse separada de la lógica del grafo.

A. Código completo

Código 10: red_amigos.rkt

```
#lang racket

(define red-inicial
  '(("ana" "beto" "carla")
    ("beto" "ana" "diego")
    ("carla" "ana" "diego" "elena")
    ("diego" "beto" "carla" "fer")
    ("elena" "carla")
    ("fer" "diego")
    ("gael")))

(define (nombre nodo) (car nodo))
(define (amigos-de nodo) (cdr nodo))

(define (buscar red persona)
  (cond [(null? red) #f]
        [(string=? (nombre (car red)) persona) (car red)]
        [else (buscar (cdr red) persona)]))

(define (existe? red persona)
  (if (buscar red persona) #t #f))

(define (amigos red persona)
  (let ([nodo (buscar red persona)])
    (if nodo (amigos-de nodo) '())))

(define (personas red) (map nombre red))

(define (son-amigos? red a b)
  (if (member b (amigos red a)) #t #f))

(define (agregar-a lista x)
  (if (member x lista) lista (append lista (list x))))

(define (quitar-de lista x)
  (filter (lambda (y) (not (string=? y x))) lista))

(define (actualizar red persona f)
  (map (lambda (nodo)
         (if (string=? (nombre nodo) persona)
             (cons (nombre nodo) (f (amigos-de nodo)))
             nodo))
       red))
```

```
(define (modificar-lazo operacion a b)
  (lambda (red) (actualizar red a (lambda (lista) (operacion lista b)))))

(define (conectar red a b)
  ((compose (modificar-lazo agregar-a b a) (modificar-lazo agregar-a a b))
      red))

(define (desconectar red a b)
  ((compose (modificar-lazo quitar-de b a) (modificar-lazo quitar-de a b))
      red))

(define (agregar-persona red persona)
  (append red (list (list persona))))

(define (sin-nodo persona)
  (lambda (red) (filter (lambda (nodo) (not (string=? (nombre nodo)
     persona))) red)))

(define (sin-referencias persona)
  (lambda (red) (map (lambda (nodo) (cons (nombre nodo) (quitar-de (
     amigos-de nodo) persona))) red)))

(define (eliminar-persona red persona)
  ((compose (sin-referencias persona) (sin-nodo persona)) red))

(define (amigos-de-amigos red persona)
  (append-map (lambda (amigo) (amigos red amigo)) (amigos red persona)))

(define (descartar-conocidos red persona)
  (lambda (lista)
    (filter (lambda (x) (and (not (string=? x persona))
                             (not (son-amigos? red persona x))))
            lista)))

(define (en-comun red a b)
  (length (filter (lambda (x) (member x (amigos red b))) (amigos red a))))

(define (ordenar-por-comun red persona)
  (lambda (lista) (sort lista > #:key (lambda (x) (en-comun red persona x)
     ))))

(define (sugerencias-directas red persona)
  ((compose (ordenar-por-comun red persona)
            remove-duplicates
            (descartar-conocidos red persona))
   (amigos-de-amigos red persona)))

(define (sugerir red persona)
  (let ([directas (sugerencias-directas red persona)])
    (if (null? directas)
        ((descartar-conocidos red persona) (personas red))
        directas)))

(define normalizar (compose string-downcase string-trim))
```

```
(define (leer mensaje)
  (display mensaje)
  (flush-output)
  (let ([linea (read-line)])
    (if (eof-object? linea) "" (normalizar linea))))

(define (texto-amigos lista)
  (if (null? lista) "(sin amigos)" (string-join lista ", ")))

(define (mostrar-red red)
  (cond [(null? red) (void)]
        [else (printf "  ~a -> ~a\n" (nombre (car red)) (texto-amigos (
           amigos-de (car red))))
              (mostrar-red (cdr red))]))

(define (mostrar-sugerencias red persona lista i)
  (cond [(null? lista) (void)]
        [else (printf "  ~a. ~a (~a amigo(s) en común)\n" i (car lista) (
           en-comun red persona (car lista)))
              (mostrar-sugerencias red persona (cdr lista) (+ i 1))]))

(define (opcion-mostrar red)
  (displayln "\n--- Red de amigos ---")
  (mostrar-red red)
  red)

(define (pedir-conexiones red persona)
  (let ([amigo (leer "  ¿De quién es amigo? (Enter para terminar): ")])
    (cond [(string=? amigo "") red]
          [(string=? amigo persona)
           (displayln "  No puede ser amigo de sí mismo.")
           (pedir-conexiones red persona)]
          [(not (existe? red amigo))
           (displayln "  Esa persona no existe.")
           (pedir-conexiones red persona)]
          [(son-amigos? red persona amigo)
           (displayln "  Ya son amigos.")
           (pedir-conexiones red persona)]
          [else
           (printf "  ¡~a y ~a ahora son amigos!\n" persona amigo)
           (pedir-conexiones (conectar red persona amigo) persona)])))

(define (opcion-agregar red)
  (let ([persona (leer "Nombre de la nueva persona: ")])
    (cond [(string=? persona "")
           (displayln "El nombre no puede estar vacío.")
           red]
          [(existe? red persona)
           (displayln "Esa persona ya está en la red.")
           red]
          [else
           (printf "Personas en la red: ~a\n" (string-join (personas red)
              ", "))
```

```
           (pedir-conexiones (agregar-persona red persona) persona)])))

(define (elegir-sugerencia red persona lista)
  (let* ([entrada (leer "Número de la persona a agregar (Enter para no
     agregar): ")]
         [n (string->number entrada)])
    (cond [(string=? entrada "") red]
          [(and n (exact-positive-integer? n) (<= n (length lista)))
           (let ([nuevo (list-ref lista (- n 1))])
             (printf "¡~a y ~a ahora son amigos!\n" persona nuevo)
             (conectar red persona nuevo))]
          [else
           (displayln "Número no válido.")
           (elegir-sugerencia red persona lista)])))

(define (opcion-sugerir red)
  (let ([persona (leer "¿A quién le sugerimos amigos?: ")])
    (if (not (existe? red persona))
        (begin (displayln "Esa persona no existe.") red)
        (let ([lista (sugerir red persona)])
          (cond [(null? lista)
                 (displayln "No hay sugerencias: ya es amigo de todos.")
                 red]
                [else
                 (if (null? (sugerencias-directas red persona))
                     (displayln "No tiene amigos de amigos, estas son
                        todas las personas de la red:")
                     (printf "Amigos de los amigos de ~a:\n" persona))
                 (mostrar-sugerencias red persona lista 1)
                 (elegir-sugerencia red persona lista)])))))

(define (opcion-romper red)
  (let ([a (leer "Primera persona: ")])
    (if (not (existe? red a))
        (begin (displayln "Esa persona no existe.") red)
        (begin
          (printf "Amigos de ~a: ~a\n" a (texto-amigos (amigos red a)))
          (let ([b (leer "¿Con quién rompe la amistad?: ")])
            (if (son-amigos? red a b)
                (begin (printf "~a y ~a ya no son amigos.\n" a b)
                       (desconectar red a b))
                (begin (displayln "Esas personas no son amigas.") red)))))
                   ))

(define (opcion-eliminar red)
  (let ([persona (leer "¿A quién eliminamos de la red?: ")])
    (if (existe? red persona)
        (begin (printf "~a fue eliminado(a) de la red.\n" persona)
               (eliminar-persona red persona))
        (begin (displayln "Esa persona no existe.") red))))

(define opciones
  (list (cons "1" opcion-mostrar)
        (cons "2" opcion-agregar)
```

```
        (cons "3" opcion-sugerir)
        (cons "4" opcion-romper)
        (cons "5" opcion-eliminar)))

(define (menu red)
  (displayln "\n========= RED DE AMIGOS =========")
  (displayln "1. Mostrar red")
  (displayln "2. Agregar persona")
  (displayln "3. Sugerir amigos")
  (displayln "4. Romper amistad")
  (displayln "5. Eliminar persona")
  (displayln "6. Salir")
  (let* ([opcion (leer "Elige una opción: ")]
         [accion (assoc opcion opciones)])
    (cond [(string=? opcion "6") (displayln "¡Hasta luego!")]
          [accion (menu ((cdr accion) red))]
          [else (displayln "Opción no válida.") (menu red)])))

(menu red-inicial)
```